

Introduction à la programmation

Table des matières

I. Contexte	3
II. Découvrir l'émergence de la programmation	3
A. Découvrir l'émergence de la programmation	3
B. Exercice : Quiz	7
III. Le fonctionnement	8
A. Le fonctionnement	8
B. Exercice : Quiz	13
IV. Essentiel	14
V. Auto-évaluation	15
A. Exercice	15
B. Test	15
Solutions des exercices	16

I. Contexte

Durée : 1 h

Prérequis : aucun

Contexte

Dans le monde professionnel d'aujourd'hui, la programmation est une compétence essentielle, quel que soit le domaine dans lequel vous évoluez. Que vous aspiriez à devenir développeur de logiciels, concepteur de sites web ou même expert en analyse de données, la programmation sera un outil puissant à maîtriser.

Dans ce cours, vous découvrirez les bases de la programmation et de l'algorithmie, les fondements sur lesquels repose tout le processus de création de logiciels et d'applications informatiques.

Nous commencerons par explorer l'histoire et les origines de la programmation. Ensuite, nous plongerons dans la compréhension de ce qu'est réellement la programmation. Nous étudierons le rôle du programmeur et la définition précise d'un programme informatique.

Vous aurez ensuite l'occasion d'explorer les premiers langages de programmation et de comprendre comment les programmes sont créés pour répondre à des besoins spécifiques. Vous découvrirez aussi l'importance des algorithmes dans la programmation moderne et comment ils sont utilisés pour optimiser les processus et prendre des décisions logiques.

Enfin, nous passerons en revue le fonctionnement global de la programmation, en examinant les différents niveaux de langages de programmation. Vous comprendrez les principes fondamentaux qui guident l'exécution d'un programme et les étapes clés du processus de programmation.

Préparez-vous à explorer le monde fascinant de la programmation et à acquérir des compétences qui vous seront précieuses tout au long de votre parcours professionnel. Prêt à plonger dans l'univers de la programmation ? Commençons dès maintenant cette aventure passionnante !

II. Découvrir l'émergence de la programmation

A. Découvrir l'émergence de la programmation

Définition Qu'est-ce que la programmation ?

Commençons par le commencement, en termes simples, la programmation c'est l'art de donner des instructions à un ordinateur pour qu'il réalise des tâches spécifiques. Programmer, c'est développer un programme, c'est-à-dire donner des étapes à suivre à une machine pour qu'elle accomplisse une action précise. La programmation permet de contrôler le comportement de cette machine en lui disant quoi faire et comment le faire.

Définition Qu'est-ce qu'un programme ?

Un programme est un ensemble de codes qui accomplit une tâche spécifique. Par exemple, un programme peut être un navigateur web, un logiciel de traitement de texte ou encore un jeu vidéo. Pensez à une recette de cuisine. Un programme informatique peut être comparé à une recette.

Dans une recette, vous avez une série d'instructions détaillées sur la manière de préparer un plat spécifique. Chaque étape de la recette décrit ce qu'il faut faire, dans quel ordre et avec quels ingrédients. En suivant les instructions de la recette, vous obtenez le résultat souhaité, à savoir un délicieux repas.

De manière similaire, un programme informatique est une série d'instructions écrites dans un langage de programmation spécifique. Ces instructions indiquent à l'ordinateur les étapes à suivre pour effectuer une tâche particulière. Le programme peut contenir des calculs, des conditions, des boucles et d'autres commandes qui permettent à l'ordinateur de prendre des décisions et d'effectuer des actions.

Le programme informatique, tout comme une recette, fournit une séquence précise d'instructions pour atteindre un résultat spécifique. Il permet à l'ordinateur d'exécuter des tâches de manière automatique et précise, en utilisant les règles et les logiques définies par le programmeur.

Ainsi, un programme informatique est comme une recette qui guide l'ordinateur dans la réalisation d'une tâche spécifique, en lui fournissant les instructions nécessaires pour obtenir le résultat attendu.

Rôle du programmeur

Quel est le rôle du programmeur ? Prenons un exemple.

Dans une cuisine, le cuisinier est celui qui crée et suit la recette pour préparer un plat. Il choisit les ingrédients appropriés, suit les étapes de la recette, ajuste les quantités si nécessaire et fait en sorte que le plat soit préparé de la meilleure manière possible. De manière similaire, le programmeur est celui qui crée un programme informatique en utilisant un langage de programmation. Le programmeur est comme un chef de cuisine qui conçoit la recette.

Le rôle du programmeur est de transformer une idée ou une tâche en une séquence d'instructions compréhensibles par l'ordinateur. Il réfléchit à la logique du programme, identifie les étapes nécessaires et écrit les instructions dans un langage de programmation spécifique.

Le programmeur choisit les bons outils, tels que les langages de programmation, les bibliothèques de codes et les frameworks (composants prédéfinis qui donnent une base pour construire du code), pour réaliser la tâche souhaitée. Son objectif est d'écrire un code clair et bien structuré afin que l'ordinateur puisse le comprendre et l'exécuter correctement.

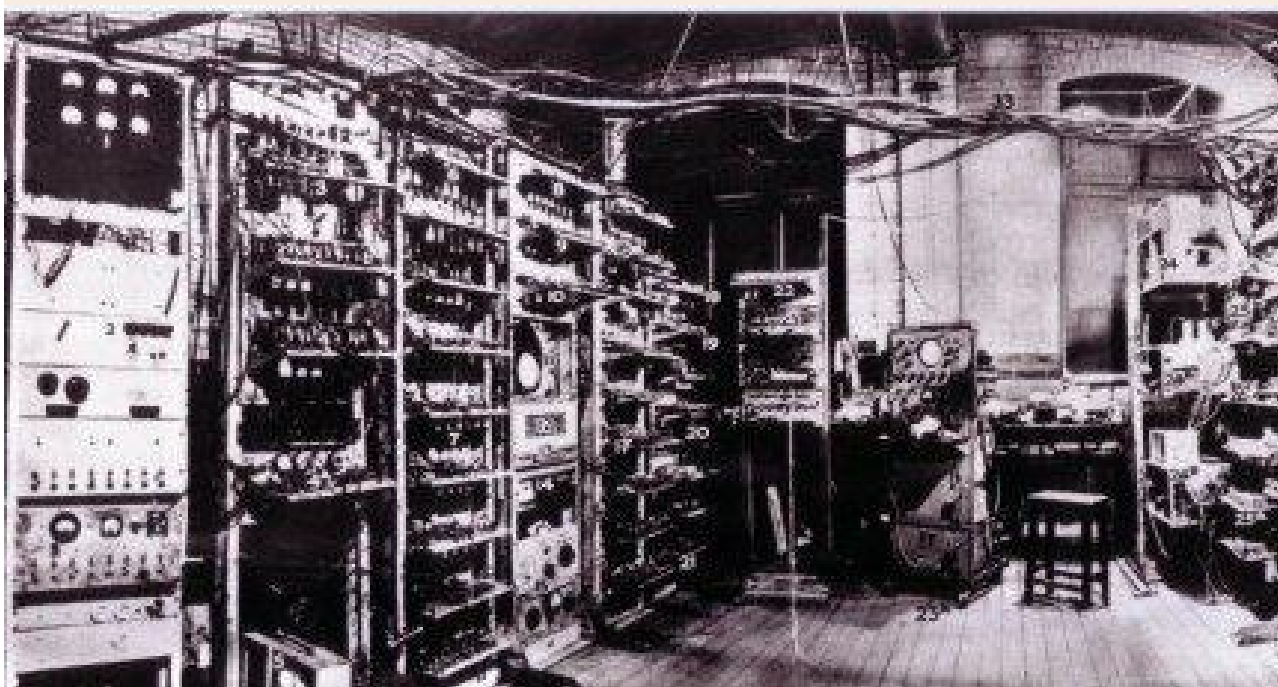
Tout comme un cuisinier peut ajuster les ingrédients et les techniques en fonction de ses préférences, un programmeur peut également apporter des modifications à son code pour obtenir le résultat souhaité. Il teste et débogue le programme pour s'assurer qu'il fonctionne correctement et répond aux attentes. Pour compléter ces propos, prenons un peu d'avance. Voici un tableau présentant une liste de fonctionnalités couramment utilisés par les programmeurs pour rendre leur code plus clair :

Fonctionnalité	Fonction
Indentations	Utilisation de l'espacement pour structurer et organiser le code
Commentaires	Ajout de notes explicatives dans le code pour expliquer son fonctionnement et faciliter la compréhension
Nommage des variables	Utilisation de noms significatifs pour les variables afin de rendre leur intention claire
Documentation	Rédaction de documentation détaillée pour expliquer l'utilisation et le fonctionnement du code
Découpage en fonctions	Division du code en fonctions logiques et réutilisables pour faciliter la compréhension et la maintenance
Gestion des espaces	Utilisation appropriée des espaces pour améliorer la lisibilité et la compréhension du code
Utilisation de constantes	Définition de constantes pour rendre le code plus lisible et éviter la répétition de valeurs
Utilisation de bibliothèques	Intégration de bibliothèques ou de modules prédéfinis pour réutiliser des fonctionnalités existantes
Organisation du code	Structuration du code en blocs logiques, en utilisant des en-têtes et des séparateurs clairs

Fonctionnalité	Fonction
Éviter la duplication	Suppression des doublons de code pour maintenir la cohérence et faciliter les mises à jour
Utilisation de conventions	Respect des conventions de codage pour garantir une cohérence dans le style et la syntaxe

En résumé, le rôle du programmeur est de concevoir, écrire et mettre en œuvre un programme informatique en utilisant un langage de programmation. Il est responsable de transformer une idée en une séquence d'instructions logiques pour permettre à l'ordinateur d'effectuer des tâches spécifiques de manière automatique et précise. Le programmeur est comme un chef qui crée une recette pour obtenir un plat délicieux, mais dans le monde de l'informatique.

Complément La première machine à programme enregistré



Source : [ZDNet.fr](https://www.zdnet.fr)¹

Vous faites actuellement connaissance avec Baby née en 1948, une bête de course de 128 octets de mémoire et une vitesse d'opération de 700 opérations par seconde ! À l'époque de Baby, la création des programmes était bien plus complexe. Voyons pourquoi.

Les premiers langages

Il existe différents langages de programmation, tout comme il existe différentes langues dans le monde. Chaque langage de programmation a ses propres règles et conventions pour écrire le code. C'est comme si vous choisissiez de parler français, anglais ou espagnol pour communiquer avec les autres. Les langages de programmation permettent aux développeurs de traduire leurs idées en code compréhensible par l'ordinateur.

¹ <https://www.zdnet.fr/actualites/le-premier-ordinateur-a-programme-enregistre-fete-ses-60-ans-39381863.htm>

Cependant, les tout premiers langages ne laissaient pas beaucoup de choix à nos amis programmeurs. Il fallait beaucoup de courage et de temps pour développer un programme. Voyons pourquoi en faisant un petit point historique :

1. Langage machine :

Au tout début, les ordinateurs ne comprenaient que le langage machine, qui est une série de 0 et de 1 représentant les instructions directement exécutées par le processeur. Ce langage était très bas niveau et difficile à utiliser, car chaque instruction devait être écrite manuellement en binaire.

Nous verrons plus tard ce que « *Bas niveau* » et « *Haut niveau* » implique.

Pour l'exemple, voici à quoi ressemblerait notre programme précédent en langage binaire :

```
1 01001000 01100101 01101100 01101100 01101111 00100000 01110111 01101111 01110010 01101100
   01100100
2 00100000 01101111 01100110 00100000 01110100 01101000 01101001 01110011 00100000 01101101
   01100001 01100011 01101000 01101001 01101110 01100101
```

Et oui c'est bien ce que lit notre machine ! Comment cela se passe-t-il en interne ?

En fait, chaque groupe de 8 bits représente un caractère en utilisant le code ASCII. Le programme en langage binaire charge les valeurs binaires correspondantes des caractères « *Bienvenue dans notre programme* » dans la mémoire et les affiche à l'aide des instructions appropriées spécifiques à l'architecture du processeur.

2. Assembleur :

Une fois que nous avons connaissance de la complexité du langage binaire, nous comprenons l'intérêt de faciliter la programmation. C'est pourquoi les langages d'assemblage ont été créés. L'assembleur permettait de remplacer les codes binaires par des séquences de caractères compréhensibles (ou mnémoniques) par les programmeurs. Chaque instruction en assembleur correspondait à une instruction spécifique du langage machine. Cela rendait la programmation plus facile, mais restait très lié à l'architecture matérielle de l'ordinateur.

Voici à quoi ressemble un programme en langage d'assemblages :

```
; Début du programme
ORG 1000h      ; Adresse de départ du programme

JMP start     ; Saut vers le début du programme

message DB 'Bienvenue dans notre programme', 0

start:
MOV AH, 9      ; Charge la valeur 9 dans le registre AH pour afficher une chaîne de caractères
MOV DX, message ; Charge l'adresse de la chaîne de caractères dans le registre DX
INT 21h        ; Appel du service d'interruption 21h pour afficher la chaîne

MOV AH, 4Ch    ; Charge la valeur 4Ch dans le registre AH pour terminer le programme
INT 21h        ; Appel du service d'interruption 21h pour terminer le programme

; Fin du programme
```

Ce programme utilise le langage Assembly. Il commence par définir l'adresse de départ du programme et utilise des instructions spécifiques à cette architecture pour afficher la chaîne de caractères « *Bienvenue dans notre programme* » à l'aide du service d'interruption 21 h. Enfin, le programme se termine en appelant à nouveau le service d'interruption 21 h avec une valeur spécifique pour indiquer la fin du programme.

Voici un récapitulatif historique :

Période	Langages de programmation	Principales caractéristiques
Années 1940-1950	Langage machine	Utilisation de codes binaires pour représenter les instructions de l'ordinateur

Période	Langages de programmation	Principales caractéristiques
Années 1950-1960	Assembleur	Remplacement des codes binaires par des mnémoniques pour faciliter la programmation

Rassurez-vous, vous n'aurez pas besoin de maîtriser ces langages ! Nous les abordons ensemble pour comprendre ce qu'il se passe au plus proche de la machine lorsque nous programmons un logiciel ou une autre solution.

B. Exercice : Quiz

[solution n°1 p.17]

Question 1

Quel est l'ancêtre de la programmation ?

- ☐ Le programmeur
- ☐ Le calculateur
- ☐ La calculatrice

Question 2

Quelle est la particularité de l'ordinateur Baby ?

- ☐ C'est le tout premier ordinateur
- ☐ C'est le premier ordinateur à programme enregistré
- ☐ C'est le premier ordinateur portable

Question 3

Quel est le seul langage de programmation que comprend un ordinateur ?

- ☐ Le langage binaire aussi appelé le langage machine
- ☐ L'assembleur
- ☐ Un langage de haut-niveau

Question 4

Quel est le rôle du programmeur ?

- ☐ Gérer un emploi du temps
- ☐ Écrire des instructions pour un ordinateur
- ☐ Dépanner des ordinateurs

Question 5

Qu'est-ce qu'un programme ?

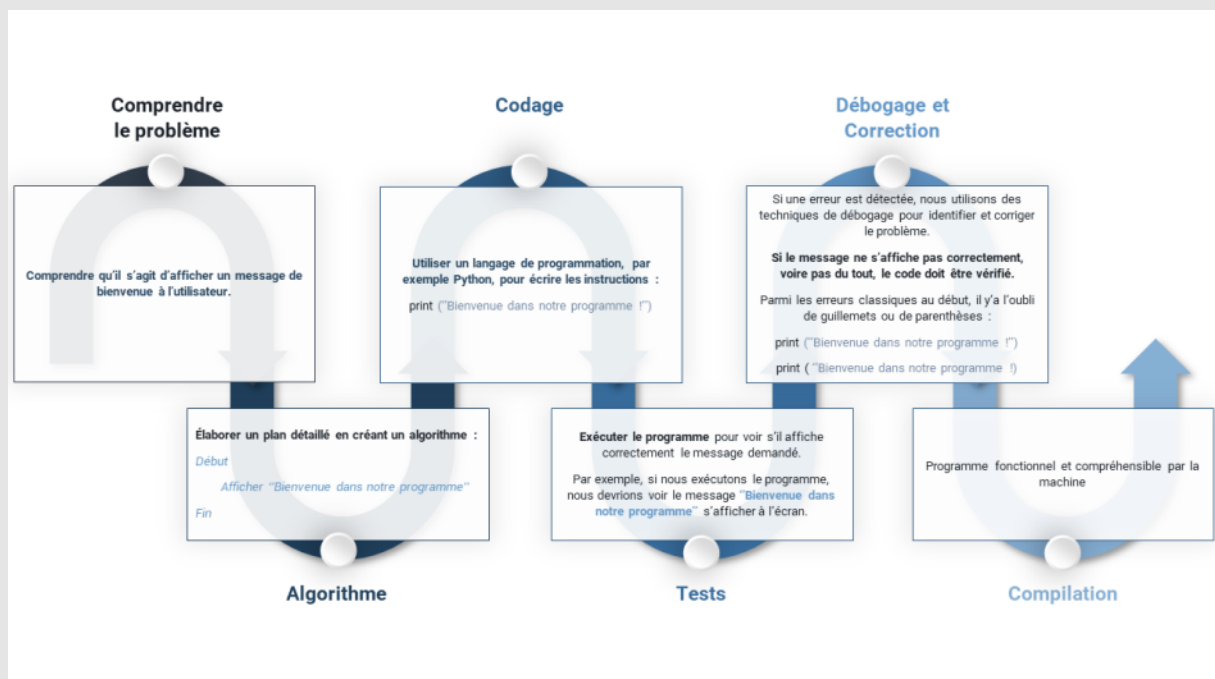
- ☐ Une série de chiffres et de lettres aléatoires
- ☐ Un ensemble d'instructions pour un ordinateur
- ☐ Une pièce de matériel informatique

III. Le fonctionnement

A. Le fonctionnement

Méthode Création d'un programme

Imaginons que nous voulons créer un programme qui affiche un message de bienvenue à l'utilisateur. Voici comment nous pourrions expliquer les actions à effectuer très simplement :



À noter sur ce schéma que la notion de compilation n'est pas expliquée. Pas d'inquiétude, nous y reviendrons plus tard.

Définition L'algorithme

Un algorithme est une séquence d'instructions logiques pour résoudre un problème ou accomplir une tâche. Il peut être comparé à un ensemble d'étapes détaillées, comme la recette d'un sandwich, qui doit être suivi dans l'ordre pour obtenir le résultat souhaité. L'algorithme fournit une structure et une logique pour résoudre des problèmes, indépendamment du langage de programmation utilisé pour l'implémenter.

Exemple

Imaginons que nous souhaitons expliquer à quelqu'un comment faire un sandwich. Voici une succession d'étapes qui pourraient nous aider à former notre algorithme :

- Étape 1 : prenez deux tranches de pain.
- Étape 2 : étalez du beurre sur une des tranches de pain.

- Étape 3 : ajoutez une tranche de jambon sur le beurre.
- Étape 4 : placez une feuille de laitue sur le jambon.
- Étape 5 : ajoutez une tranche de fromage sur la laitue.
- Étape 6 : posez la deuxième tranche de pain sur le fromage.
- Étape 7 : appuyez légèrement sur le sandwich pour le comprimer.

Dans cet exemple, l'algorithme est une séquence d'étapes logiques qui décrit comment assembler les différents ingrédients pour obtenir un sandwich. Chaque étape est clairement définie et doit être suivie dans l'ordre pour obtenir le résultat attendu, c'est-à-dire un sandwich prêt à être dégusté.

L'algorithme du sandwich peut être comparé à un plan détaillé pour réaliser une tâche spécifique. Il ne dépend pas d'un langage particulier, car il peut être lu et compris par n'importe qui, qu'il comprenne ou non une langue donnée.

Définition Le code

Par définition, le code informatique est un ensemble d'instructions écrites dans un langage de programmation pour indiquer à l'ordinateur comment effectuer une tâche spécifique. Le code informatique est à distinguer absolument de l'algorithme.

Nous avons vu qu'un **algorithme** est une séquence d'étapes ou d'instructions logiques utilisées pour résoudre un problème. C'est une sorte de plan ou de stratégie pour atteindre un résultat souhaité. En d'autres termes, un algorithme décrit la logique de la résolution d'un problème, mais il **n'est pas lié à un langage de programmation spécifique**.

Le code informatique, quant à lui, est la traduction de cet algorithme en langage compréhensible par l'ordinateur. C'est l'ensemble d'instructions précises écrites dans un langage de programmation donné pour implémenter l'algorithme et résoudre le problème. Le code est plus concret et spécifique, car il utilise une syntaxe et des conventions propres à un langage de programmation.

Le code est ce qui donne vie à votre idée et permet à l'ordinateur de faire ce que vous voulez. Il est également étroitement lié au programme et aux langages de programmation. Le code informatique est la base du programme, car il contient les instructions nécessaires pour que le programme fonctionne correctement.

Définition Le compilateur

Le rôle du compilateur dans la création d'un programme informatique est essentiel. Le compilateur est un logiciel qui transforme le code source, écrit dans un langage de programmation compréhensible par les humains, en un code exécutable par l'ordinateur, appelé code machine. Il agit comme un traducteur entre le langage de programmation et l'ordinateur.



Exemple

Pour illustrer cela, prenons un exemple. Supposons que nous voulions écrire un programme simple en langage C qui additionne deux nombres saisis par l'utilisateur et affiche le résultat à l'écran. Voici à quoi ressemblerait le code source :

```
1 #include <stdio.h>
2
3 int main() {
4     int a, b, somme;
5
6     printf("Entrez le premier nombre : ");
7     scanf("%d", &a);
8
9     printf("Entrez le deuxième nombre : ");
10    scanf("%d", &b);
11
12    somme = a + b;
13
14    printf("La somme est : %d\n", somme);
15
16    return 0;
17 }
```

Une fois le code source créé, nous devons passer par une étape de compilation pour transformer ce code source en un fichier exécutable. Le compilateur prend en charge cette tâche, il effectue alors plusieurs actions :

- Il vérifie la syntaxe du code pour s'assurer qu'il est correctement écrit. Il identifie les erreurs de syntaxe telles que les parenthèses manquantes ou les points-virgules oubliés.
- Il traduit le code source en langage machine, qui est compréhensible par l'ordinateur. Il effectue une série de transformations et d'optimisations pour générer un fichier exécutable.



Le fichier exécutable résultant peut être directement exécuté sur l'ordinateur. Lorsque nous exécutons ce fichier, le programme additionne les deux nombres saisis par l'utilisateur et affiche le résultat à l'écran.

Ainsi, le compilateur joue un rôle essentiel dans la création d'un programme informatique en traduisant le code source en un format exécutable. Il permet aux programmeurs d'écrire des programmes dans des langages de haut niveau plus conviviaux, tout en garantissant que l'ordinateur peut exécuter ces programmes de manière efficace et précise.

Remarque

Les erreurs

Lors des tests, si le programme nous renvoi des erreurs, ce qui arrive assurément, est-ce forcément de la faute du programmeur ? Oui. Pas de place à l'orgueil en programmation, la machine aura toujours raison par rapport à l'humain, s'il y a une erreur il faudra alors la rechercher, la comprendre puis la corriger.

Programmation de haut et bas niveau

La programmation peut être abordée sous différentes formes et niveaux de complexité. Deux concepts importants à comprendre sont la programmation de haut niveau et la programmation de bas niveau.

Avec le développement des ordinateurs et l'essor de la programmation, les langages de haut niveau ont été introduits pour faciliter le processus de développement de logiciels. Ces langages permettent aux programmeurs d'écrire des instructions plus abstraites et plus proches du langage humain. Ils offraient des structures de contrôle plus avancées, comme les boucles et les conditions, ainsi que des fonctionnalités de gestion de la mémoire plus sophistiquées.

Au fil du temps, les langages de programmation de haut niveau ont continué à évoluer pour répondre aux besoins changeants des programmeurs. De nouveaux langages ont été créés, offrant des fonctionnalités spécifiques différentes. Certains langages se sont concentrés sur la simplicité et la lisibilité du code, tandis que d'autres ont mis l'accent sur la performance et l'efficacité.

Les évolutions des langages de programmation ont été motivées par plusieurs facteurs. D'un côté, les besoins des programmeurs ont évolué au fur et à mesure que les applications informatiques devenaient de plus en plus complexes. Les langages de programmation devaient donc offrir des fonctionnalités plus avancées pour répondre à ces besoins croissants.

D'un autre côté, les avancées technologiques ont rendu les ordinateurs plus puissants et capables de traiter des tâches plus complexes. Les langages de programmation ont donc dû évoluer pour tirer parti de ces nouvelles capacités matérielles.

D'autre part, la programmation de bas niveau implique l'utilisation de langages plus proches du langage machine, tels que le langage d'assemblage vu précédemment. Ces langages sont plus complexes et nécessitent une compréhension détaillée de l'architecture matérielle de la machine. La programmation de bas niveau permet un contrôle plus précis sur le matériel, mais elle est généralement plus difficile à apprendre et à utiliser.

Dans la pratique, la programmation de haut niveau est souvent privilégiée car elle permet aux programmeurs de se concentrer sur la logique du programme plutôt que sur les détails techniques. Cependant, la programmation de bas niveau est parfois nécessaire pour des tâches spécifiques où un contrôle précis du matériel est requis.

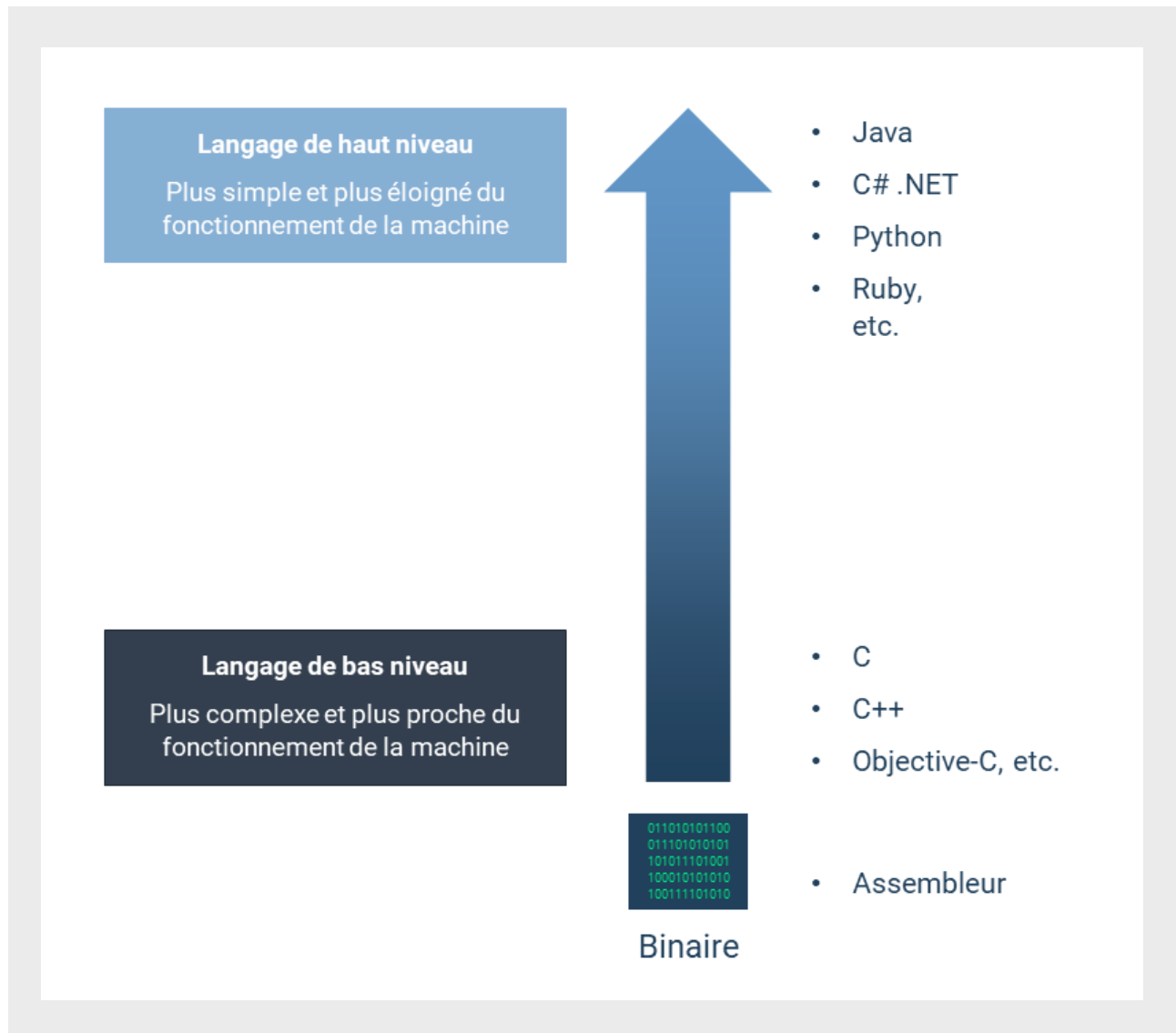
Il est important de noter que la programmation de haut niveau et la programmation de bas niveau ne sont pas mutuellement exclusives. Les programmeurs peuvent utiliser des langages de haut niveau pour écrire la majeure partie de leur code et incorporer des éléments de programmation de bas niveau lorsque cela est nécessaire.

Exemple Classification des langages

Voici un tableau présentant une liste, bien sûr non exhaustive, de certains langages de haut et bas niveau accompagnés d'une description.

Langages de bas niveau	Description	Langages de haut niveau	Description
Langage d'assemblage	Permet de programmer directement sur le matériel en utilisant des instructions proches du langage machine.	Python	Langage de haut niveau populaire, facile à lire et à comprendre. Adapté au développement rapide d'applications.
Langage machine	Langage binaire compréhensible par le processeur. Utilisé pour écrire des programmes directement exécutables par la machine.	Java	Langage de haut niveau utilisé pour le développement d'applications, en particulier pour les applications Android.
C	Permet une programmation de bas niveau tout en offrant une certaine abstraction pour faciliter le développement. Utilisé pour le développement de systèmes d'exploitation et de pilotes.	C#	Langage de haut niveau basé sur C, utilisé pour le développement de logiciels système, de jeux et d'applications haute performance.
Ada	Langage utilisé dans les applications critiques nécessitant une forte fiabilité et une sécurité élevée, notamment dans les domaines de l'aérospatiale et de la défense.	JavaScript	Langage de haut niveau utilisé pour le développement web, permettant d'ajouter des fonctionnalités interactives aux sites internet.
Rust	Langage de programmation système moderne, conçu pour garantir la sécurité et la performance. Utilisé pour le développement d'applications fiables et performantes.	Ruby	Langage de haut niveau utilisé pour le développement web et l'automatisation de tâches. Connue pour sa simplicité et son élégance.

Enfin, en guise de résumé, voici un graphique présentant une classification simple des langages de programmation.



B. Exercice : Quiz

[solution n°2 p.18]

Question 1

Python est un langage de bas niveau.

- ☐ Vrai
- ☐ Faux

Question 2

À quoi correspond ces lignes :

Début

Variable message

message ← "bonjour"

Afficher message

Fin

- ☐ Un algorithme
- ☐ Du code informatique
- ☐ Du langage machine

Question 3

Quelles étapes permettent le traitement des erreurs de code ?

- ☐ Comprendre le problème et la planification
- ☐ Le codage et la compilation
- ☐ Le codage, les tests, le débogage et la correction

Question 4

Les erreurs de codes sont souvent dû à la machine.

- ☐ Vrai
- ☐ Faux

Question 5

Quelle est la première étape de la création d'un programme ?

- ☐ Le codage
- ☐ La planification
- ☐ Chercher à comprendre le problème

IV. Essentiel

Alors que retenir ? La programmation consiste à écrire du code dans un langage de programmation pour créer des programmes informatiques qui réalisent des tâches précises. C'est une pratique où vous donnez vie à vos idées et guidez l'ordinateur à travers vos instructions.

Ce processus tient ses racines des calculateurs, les premières machines conçues pour automatiser les opérations arithmétiques. L'ordinateur Baby, connu également sous le nom de Manchester Mark 1, est considéré comme le premier ordinateur à programme enregistré, car il pouvait stocker les instructions et les données dans sa mémoire.

Il existe différents langages de programmation, dont le langage binaire, compris par les ordinateurs, mais difficile à utiliser pour les humains. Les langages de haut niveau, tels que Java, Python et C#, sont plus abstraits et conviviaux pour les programmeurs, ce qui les rend plus utilisés que les langages de bas niveau, tels que le langage machine.

Le processus de programmation comprend plusieurs étapes : la compréhension du problème à résoudre, la planification de l'algorithme, l'écriture du code source dans un langage de programmation, les tests et la correction des erreurs éventuelles. Pour traduire le code source en code machine exécutable, on utilise un compilateur.

Enfin, dans le fonctionnement global de la programmation, le processeur joue un rôle central. Grâce à sa mémoire, il exécute les instructions du programme, traite les données et produit les résultats attendus par le programmeur.

Il est important de comprendre que la programmation nécessite à la fois une logique et une créativité pour résoudre des problèmes et créer des solutions informatiques. En acquérant des compétences en programmation, vous serez en mesure de développer des logiciels, des applications, des sites web et bien d'autres choses encore.

V. Auto-évaluation

A. Exercice

Vous participez à un concours d'éloquence et pour gagner le premier prix vous devez présenter un exposé sur une thématique précise. Vous avez choisi les origines et les bases de la programmation.

Vous devez maintenant préparer votre exposé, vous souhaitez mettre l'accent sur les premiers langages de programmation et la distinction entre la programmation de haut niveau et de bas niveau. Vous devez alors répondre à ces questions :

Question 1

[solution n°3 p.19]

Quels étaient les premiers langages de programmation et en quoi étaient-ils différents de la programmation moderne ?

Question 2

[solution n°4 p.19]

Quelle est la distinction entre la programmation haut et bas niveau et pourquoi la programmation de haut niveau est plus couramment utilisée ?

B. Test

Exercice 1 : Quiz

[solution n°5 p.20]

Question 1

Qu'est-ce que la programmation ?

- ☐ La création d'un langage de programmation
- ☐ L'écriture d'un code source pour réaliser des tâches spécifiques
- ☐ L'optimisation des performances d'un programme

Question 2

Quel est le rôle du compilateur ?

- ☐ Traduire le code source en code machine exécutable
- ☐ Analyser et déboguer le code source
- ☐ Optimiser les performances du programme

Question 3

Quelles sont les étapes clés de la programmation dans l'ordre ?

- ☐ Comprendre le problème > planification > codage > tests > déboguer et corriger > compilation
- ☐ Comprendre le problème > compilation > Déboguer et corriger > tests > codage > planification
- ☐ Comprendre le problème > planification > tests > déboguer et corriger > compilation > codage

Question 4

Le langage de bas niveau est plus utilisé que le langage de haut niveau.

- ☐ Vrai
- ☐ Faux

Question 5

Quel rôle a le processeur dans le fonctionnement global de la programmation ?

- ☐ Exécuter les instructions du programme
- ☐ Traduire le code source en code machine
- ☐ Optimiser les performances du programme

Solutions des exercices

Exercice p. 7 Solution n°1**Question 1**

Quel est l'ancêtre de la programmation ?

- ☐ Le programmeur
- ☒ Le calculateur
- ☐ La calculatrice

 L'ancêtre de la programmation est le calculateur. Les premiers calculateurs étaient des machines mécaniques conçues pour effectuer des opérations arithmétiques automatiquement. Bien qu'ils ne soient pas aussi sophistiqués que les ordinateurs modernes, ces calculateurs ont jeté les bases de la programmation en permettant l'automatisation des calculs et en utilisant des mécanismes pour exécuter des séquences d'instructions.

Question 2

Quelle est la particularité de l'ordinateur Baby ?

- ☐ C'est le tout premier ordinateur
- ☒ C'est le premier ordinateur à programme enregistré
- ☐ C'est le premier ordinateur portable

 La particularité de l'ordinateur Baby, également connu sous le nom de Manchester Mark 1, est qu'il est considéré comme le premier ordinateur à programme enregistré. Cela signifie qu'il était capable de stocker les instructions et les données dans sa mémoire, ce qui lui permettait d'exécuter des programmes enregistrés. Avant l'ordinateur Baby, les ordinateurs nécessitent une réécriture manuelle des instructions pour chaque tâche.

Question 3

Quel est le seul langage de programmation que comprend un ordinateur ?

- ☒ Le langage binaire aussi appelé le langage machine
- ☐ L'assembleur
- ☐ Un langage de haut-niveau

 Le seul langage de programmation que comprend un ordinateur est le langage binaire, également appelé le langage machine. Les ordinateurs sont construits autour de circuits électroniques qui comprennent les instructions et les données sous forme binaire, c'est-à-dire des 0 et des 1. Les autres langages de programmation, tels que l'assembleur et les langages de haut-niveau, doivent être traduits en langage machine pour être exécutés par l'ordinateur.

Question 4

Quel est le rôle du programmeur ?

- ☐ Gérer un emploi du temps
- ☒ Écrire des instructions pour un ordinateur
- ☐ Dépanner des ordinateurs

- Q Le rôle du programmeur est d'écrire des instructions pour un ordinateur. Les programmeurs utilisent des langages de programmation pour créer des séquences d'instructions qui indiquent à l'ordinateur comment exécuter une tâche spécifique. Ils peuvent développer des logiciels, des applications, des sites web et bien d'autres choses en écrivant du code compréhensible par la machine.

Question 5

Qu'est-ce qu'un programme ?

- ☐ Une série de chiffres et de lettres aléatoires
- ☒ Un ensemble d'instructions pour un ordinateur
- ☐ Une pièce de matériel informatique

- Q Un programme est un ensemble d'instructions pour un ordinateur. Il s'agit d'un ensemble structuré d'instructions écrites dans un langage de programmation spécifique. Un programme indique à l'ordinateur les actions à effectuer, les calculs à effectuer et les opérations à exécuter pour accomplir une tâche donnée. Il peut être exécuté par l'ordinateur pour produire un résultat attendu.

Exercice p. 13 Solution n°2

Question 1

Python est un langage de bas niveau.

- ☐ Vrai
- ☒ Faux

- Q Python est un langage de haut niveau, ce qui signifie qu'il est conçu pour être facilement compréhensible par les humains et offre une abstraction élevée par rapport au fonctionnement interne de la machine.

Question 2

À quoi correspondent ces lignes :

Début

Variable message

message ← "bonjour"

Afficher message

Fin

- ☒ Un algorithme
- ☐ Du code informatique
- ☐ Du langage machine

- Q Ces lignes correspondent à un algorithme. Elles déclarent une variable "message", lui attribuent la valeur "bonjour" et l'affichent. L'ensemble des instructions est structuré et logique pour accomplir cette tâche précise. Il ne s'agit ni de code informatique spécifique ni de langage machine.

Question 3

Quelles étapes permettent le traitement des erreurs de code ?

- ☐ Comprendre le problème et la planification
 - ☐ Le codage et la compilation
 - ☒ Le codage, les tests, le débogage et la correction
-  Après avoir écrit le code, il est important de le tester pour identifier les erreurs éventuelles. Le processus de débogage consiste à rechercher et corriger ces erreurs. Ensuite, le code corrigé peut être testé de nouveau pour s'assurer qu'il fonctionne correctement.

Question 4

Les erreurs de codes sont souvent dû à la machine.

- ☐ Vrai
- ☒ Faux

 Les erreurs de code sont généralement dues à des erreurs humaines, telles que des erreurs de syntaxe, de logique ou de conception. La machine exécute le code tel qu'il est écrit, mais si le code contient des erreurs, il peut entraîner des comportements indésirables ou des plantages.

Question 5

Quelle est la première étape de la création d'un programme ?

- ☐ Le codage
- ☐ La planification
- ☒ Chercher à comprendre le problème

 Avant de commencer à coder, il est essentiel de bien comprendre le problème que l'on souhaite résoudre. Cela implique de définir clairement ce que l'on attend de l'ordinateur et de comprendre les exigences et les contraintes du problème. La planification et la définition de l'algorithme viennent après cette première étape de compréhension.

p. 15 Solution n°3

Les débuts de la programmation remontent aux premiers calculateurs, conçus pour effectuer des opérations arithmétiques automatiquement. Cependant, à cette époque, la programmation était réalisée de manière manuelle, nécessitant une réécriture des instructions pour chaque tâche.

La machine ne comprenant que le langage binaire, aussi appelé langage machine, le nombre d'experts pouvant interagir avec elle était très limité. C'est pourquoi l'assembleur a été créé dans les années 50 pour simplifier au programmeur la communication avec la machine.

Peu de temps après, les premiers langages de programmation de haut niveau sont apparus pour continuer de faciliter cette tâche. C'est à cette époque que des langages comme Fortran, COBOL et BASIC ont émergé. Ces premiers langages étaient plus limités en fonctionnalités et capacités par rapport à la programmation moderne. Ils ont toutefois jeté les bases pour le développement de langages de programmation plus avancés et conviviaux que nous utilisons aujourd'hui.

p. 15 Solution n°4

La programmation haut et bas niveau se différencie par le niveau d'abstraction et de proximité avec le langage machine. Le langage de bas niveau, tel que le langage machine, est très proche du langage compris par le processeur. Il est souvent utilisé pour des tâches de bas niveau nécessitant une optimisation de performance, mais il requiert une connaissance approfondie de l'architecture matérielle.

En revanche, la programmation de haut niveau utilise des langages plus abstraits, tels que Java, Python et C#. Ces langages offrent des fonctionnalités avancées et des abstractions qui facilitent le développement et la compréhension du code. Ils sont conçus pour être plus facile à utiliser pour les programmeurs.

La programmation de haut niveau est plus couramment utilisée car elle permet une plus grande productivité et une meilleure lisibilité du code. On peut néanmoins préciser que même si les langages de haut niveau sont préférés, dans certaines situations où les performances sont essentielles, la programmation de bas niveau peut être requise.

Exercice p. 15 Solution n°5

Question 1

Qu'est-ce que la programmation ?

- ☐ La création d'un langage de programmation
- ☒ L'écriture d'un code source pour réaliser des tâches spécifiques
- ☐ L'optimisation des performances d'un programme
-  La programmation consiste à écrire un code source, une série d'instructions précises, dans un langage de programmation pour créer des programmes informatiques capables d'effectuer des tâches spécifiques.

Question 2

Quel est le rôle du compilateur ?

- ☒ Traduire le code source en code machine exécutable
- ☐ Analyser et déboguer le code source
- ☐ Optimiser les performances du programme
-  Le compilateur est un logiciel qui traduit le code source écrit dans un langage de programmation (comme C, C++, Java) en code machine exécutable compréhensible par le processeur de l'ordinateur.

Question 3

Quelles sont les étapes clés de la programmation dans l'ordre ?

- ☒ Comprendre le problème > planification > codage > tests > déboguer et corriger > compilation
- ☐ Comprendre le problème > compilation > Déboguer et corriger > tests > codage > planification
- ☐ Comprendre le problème > planification > tests > déboguer et corriger > compilation > codage
-  Après avoir compris le problème et écrit un algorithme, nous écrivons le code source en utilisant un langage de programmation. Nous effectuons ensuite plusieurs tests et nous corrigeons les erreurs. Puis, le code est compilé à l'aide d'un compilateur pour le traduire en code machine exécutable.

Question 4

Le langage de bas niveau est plus utilisé que le langage de haut niveau.

☐ Vrai

☒ Faux

 Le langage de bas niveau, tel que le langage machine, est un langage de programmation très proche du langage compris par le processeur. Cependant, il est moins utilisé que les langages de haut niveau tels que Java, Python, C#, etc. Les langages de haut niveau sont plus faciles à apprendre et à utiliser, ils sont conçus pour être plus abstraits et plus conviviaux pour les programmeurs. Le langage de haut niveau est donc bien plus utilisé que le langage de bas niveau.

Question 5

Quel rôle a le processeur dans le fonctionnement global de la programmation ?

☒ Exécuter les instructions du programme

☐ Traduire le code source en code machine

☐ Optimiser les performances du programme

 Dans le contexte de la programmation, le rôle du processeur est d'exécuter les instructions du programme. Après que le code source ait été traduit en code machine par le compilateur, le processeur est responsable de l'exécution des instructions une par une, traitant les données et produisant les résultats attendus. En d'autres termes, le processeur est le composant qui réalise le travail effectif demandé par le programme.